

# Dynamic Actuarial Functions under Time-Dependent Policy Behavior

Jaroslav Drobek

# Overview

1

- Objective: reproduce a customer reference model inside an actuarial calculation engine using standard product infrastructure as far as possible
- Core issue: actuarial quantities were not driven by a single static mortality basis
- The model combined
  - standard annual mortality inputs,
  - a fixed mortality selection coefficient,
  - time-dependent lapse assumptions by policy year
- The task therefore was not only formula translation, but also reconstruction of a consistent computational scheme
- Additional complexity arose later from mid-term premium and payment-frequency changes

## Mathematical Foundations

- Let  $x$  be entry age,  $n$  policy term,  $t = 0, \dots, n - 1$
- The key one-year discounted survival quantity was reconstructed as

$$E_{x+t:1} = v(1 - c q_{x+t})(1 - s_t),$$

where

- $v$  is the discount factor,
  - $q_{x+t}$  is annual mortality,
  - $c$  is a fixed mortality selection coefficient,
  - $s_t$  is lapse rate in policy year  $t$
- This converts a non-standard yearly assumption set into a sequence of standard one-step building blocks

## Core Recursive Construction

- Multi-year values were obtained backward from one-year components:

$$E_{x+t:n-t} = E_{x+t:1} \cdot E_{x+t+1:n-(t+1)}$$

- The same principle extends to annuity and insurance values:

$$a_{x+t:n-t} = 1 + a_{x+t+1:n-(t+1)} E_{x+t:1}$$

$$A_{x+t:n-t} = A_{x+t:1} + A_{x+t+1:n-(t+1)} E_{x+t:1}$$

- This was the mathematical core of the solution:
  - no bespoke mortality table per duration had to be stored,
  - only standard mortality plus annual modifiers were needed,
  - all required values could be generated iteratively

## What Made the Problem Nontrivial

- The reference model was given in spreadsheet form rather than as a formal specification
- Several target quantities were defined indirectly through business formulas
- The implementation had to bridge two representations:
  - engine-side actuarial variables and functions,
  - reference-model premium and reserve quantities
- Hence the work combined:
  - reverse-engineering of formulas,
  - actuarial reformulation,
  - numerical consistency checks,
  - integration into an existing production-style code path

## Implementation Aspects

- A dedicated internal structure was introduced for precomputed actuarial quantities
- For each policy configuration, annual values of the necessary functions were stored once and reused
- Main benefit:
  - avoid repeated recomputation in pricing and reserving routines,
  - keep the logic centralized,
  - make later extensions manageable
- This turned the actuarial layer into a reusable computational component rather than scattered formula fragments

## Pricing and Reserving Integration

- After the actuarial layer was reconstructed, core calculation routines had to be aligned with reference outputs
- In particular, internal premium and reserve variables had to be derived from customer-facing quantities such as:

$$\text{Single Premium} \sim a_{x:n}, \quad \text{Reserve} \sim \text{Sum Insured}$$

- The real challenge was not the proportionality itself, but identifying which internal variables must be solved for so that the engine reproduces the external model
- In practice, this meant calibration of pricing and reserve formulas to an externally defined actuarial logic

## Extension: Contract Changes During Policy Lifetime

7

- A later extension introduced dynamic policy evolution:
  - premium changes during the contract,
  - payment-frequency changes,
  - changes in insured amount
- A static reserve trajectory was no longer sufficient
- The solution moved toward event-driven recalculation:
  - recompute future premium and reserve quantities after each change,
  - preserve change history,
  - separate persistent base policy effects from later add-on effects where needed
- This changed the problem from “formula implementation” to “state evolution under contractual events”



## Main Technical Challenges

8

1. Reconstructing hidden actuarial meaning from spreadsheet formulas
2. Converting year-specific mortality/lapse behavior into recursive actuarial functions
3. Embedding the new logic into existing pricing and reserve routines
4. Designing internal storage for reusable precomputed values
5. Preserving numerical consistency under later premium/frequency changes

## What Was Actually Solved

### Signature

A non-standard life insurance calculation was reformulated into a recursive actuarial framework that could be implemented inside an existing engine without requiring a bespoke table structure for every contract year.

- The model combined mortality selection, lapse dynamics, discounting, pricing and reserve logic in one consistent scheme
- The result was robust enough to support later contract modifications and repeated recalculation
- From a technical perspective, the key deliverable was a reusable actuarial computation layer plus its integration into the pricing/reserving pipeline

## Outcome

- Reference actuarial behavior was reproduced in a structured and formally interpretable way
- The implementation replaced spreadsheet-style dependence by explicit recursive logic
- The resulting design supported:
  - traceable actuarial quantities,
  - cleaner maintenance,
  - extension toward dynamic contract changes
- Final value of the work:
  - not just matching outputs,
  - but turning an implicit business model into an implementable mathematical component